

2026 年 7 月 11 日

摘 要

针对高温环境下的多层防护服装热传导问题, 本文建立了一维非稳态多层介质热传导模型, 并通过参数辨识与数值优化, 求解了不同工况下的最优厚度配置。

首先, 基于傅里叶热传导定律和能量守恒原理, 建立了由 I、II、III、IV 层构成的多层一维非稳态热传导偏微分方程模型。采用 Crank-Nicolson 有限差分格式进行数值离散, 以附件 2 中 5401 个有效温度数据为基准, 对模型中的外表面换热系数和皮肤等效换热系数进行了参数校准。校准结果表明, 最优外表面换热系数为 $14.0658 \text{ W}/(\text{m}^2 \cdot \text{K})$, 最优皮肤等效换热系数为 $3.2481 \text{ W}/(\text{m}^2 \cdot \text{K})$, 模型预测与实测数据的均方根误差仅为 0.0491°C , 最大绝对误差为 0.2641°C , 验证了模型的准确性。

其次, 针对子问题 2, 在环境温度 65°C 、IV 层厚度 5.5 mm 的工况下, 以 II 层厚度为单一决策变量, 以皮肤外侧温度不超过 47°C 且超过 44°C 的累计时间不超过 300 s 为约束, 采用二分法进行优化求解。结果表明, 满足约束的最小 II 层厚度为 10.4172 mm , 此时皮肤外侧最高温度为 43.9821°C , 超过 44°C 的累计时间为 0 s , 约束条件全部满足。

最后, 针对子问题 3, 在环境温度 80°C 的工况下, 以 II 层和 IV 层厚度为决策变量, 以总厚度最小化为优化目标, 采用网格搜索算法进行双变量优化。最优解为 II 层厚度 19.4181 mm 、IV 层厚度 4.8760 mm , 总厚度为 24.2942 mm 。在此配置下, 皮肤外侧最高温度为 44.5504°C , 超过 44°C 的累计时间为 300 s , 约束条件恰好满足。

关键词: 热传导模型; Crank-Nicolson 差分; 参数辨识; 多层织物优化; 有限差分法

1 问题重述

1.1 问题背景

在高温环境下工作的专业人员，如消防员、冶金工人和玻璃制造者，其生命安全与工作效率高度依赖于所穿戴的防护服装的隔热性能。传统的防护服装设计依赖于反复的物理实验和试错，这不仅耗费大量的时间和资金成本，而且难以精确量化各层材料参数对整体热防护性能的影响。因此，建立一个能够准确描述多层织物热传导过程的数学模型，并基于此模型进行参数优化，具有重要的工程应用价值。

本题所研究的专用服装由与外界环境接触的 I 层、中间的 II 层、与皮肤接触的 III 层以及 III 层与皮肤之间的空气间隙（IV 层）共四层结构组成。在给定环境温度、各层材料参数（密度、比热容、导热系数）和初始厚度的情况下，需要建立一个数学模型，模拟假人皮肤外侧（即 III 层与 IV 层交界处）的温度随时间的变化过程。

1.2 问题要求

根据上述背景，本题提出了三个递进式的子问题：

子问题 1：温度场模拟与模型验证。给定环境温度为 75°C ，II 层厚度为 6 mm，IV 层厚度为 5 mm，总工作时间为 90 分钟。要求建立热传导模型，计算并输出假人皮肤外侧的温度随时间变化的数据。附件 2 提供了同一工况下的实测温度数据，共计 5401 个有效数据点，该数据可用于验证所建模型的准确性。

子问题 2：单变量厚度优化。改变环境温度为 65°C ，IV 层厚度调整为 5.5 mm，工作时间为 60 分钟。要求在满足以下约束条件的前提下，确定 II 层的最优厚度：工作时间内假人皮肤外侧温度不得超过 47°C ，且超过 44°C 的累计时间不得超过 5 分钟。

子问题 3：双变量厚度优化。环境温度升高至 80°C ，工作时间为 30 分钟。要求同时优化 II 层和 IV 层的厚度，使得在满足与子问题 2 相同的温度约束条件下，实现某种意义上的最优设计。

1.3 问题分析

本题的核心在于建立一个准确、高效的一维非稳态热传导模型。首先，需要基于傅里叶热传导定律，建立多层介质的控制方程，并合理设定各层界面处的连续条件（温度和热流密度连续）以及边界条件（皮肤内侧恒温 37°C ，外表面与环境对流换热）。其次，需要选择合适的数值方法（如有限差分法）对偏微分方程进行离散求解，并利用附件 2 的实验数据对模型中的关键参数（如对流换热系数）进行校准。最后，在模型验证的基础上，针对不同的工况和约束条件，采用合适的优化算法（如二分法、网格搜索法）搜索最优的厚度组合。

2 模型假设

为了保证模型精度的前提下简化问题的复杂性，本文提出以下基本假设：

- 一维热传导假设：**假设热量仅沿垂直于皮肤表面的方向（即厚度方向）进行传递，忽略沿服装平面方向的横向传热。该假设基于服装的横向尺寸（长度和宽度）远大于其厚度，且环境温度分布均匀，因此热量主要沿厚度方向传递。

2. **材料均匀且各向同性假设：**假设每层材料内部是均匀且各向同性的，其热物理性质（密度 ρ_i 、比热容 c_i 、导热系数 k_i ）在材料内部处处相等，且不随温度的变化而变化。该假设基于题目给出的材料参数为常数，未提供温度依赖关系。
3. **完美热接触假设：**假设各层材料之间的界面（I-II、II-III、III-IV）接触紧密，不存在接触热阻。因此，在各层界面上，温度和热流密度均保持连续。该假设简化了界面处理，忽略了实际生产中可能存在的微小间隙和接触不良的影响。
4. **人体内部温度恒定假设：**假设假人内部温度恒定为 37°C ，即皮肤内侧的边界条件为恒温边界条件。该假设直接来源于题目中“将体内温度控制在 37°C 的假人”的描述。
5. **对流换热边界条件假设：**假设服装 I 层外表面与高温环境之间，以及 IV 层（空气层）与假人皮肤表面之间，均通过对流换热方式进行热量交换，且对流换热系数为常数。其中，外表面与环境的对流换热系数 h 和皮肤表面的等效换热系数 h_{skin} 需要通过附件 2 的实测数据辨识得到。
6. **初始温度均匀假设：**假设在初始时刻（ $t = 0$ ），整个服装系统（I、II、III、IV 层）的温度均匀一致，且等于人体内部温度 37°C 。该假设基于假人在进入高温环境前处于恒温状态的合理推断。
7. **辐射换热忽略假设：**忽略各层之间以及服装与环境之间的辐射换热。在本题的温度范围内（最高 80°C ），辐射换热量相对于对流和传导通常较小，忽略辐射可以显著简化模型。若模型精度要求更高，可考虑在后续工作中加入辐射项。

3 符号说明

本文所使用的主要符号及其含义、单位和取值范围如表1所示。

各层材料的固定热物理参数如表2所示。

4 模型的建立与求解

4.1 数据预处理与探索性分析

在建立热传导模型之前，首先对附件提供的材料参数和实验测量数据进行预处理与探索性分析。附件 1 包含 4 层材料的密度、比热容、导热系数及厚度数据，其中 I 层和 III 层的厚度为固定值（分别为 0.6 mm 和 3.6 mm），而 II 层和 IV 层的厚度标注为范围值（分别为 0.6–25 mm 和 0.6–6.4 mm），表明这两个参数为可调的设计变量。

附件 2 提供了在环境温度 75°C 、II 层厚度 6 mm、IV 层厚度 5 mm 工况下，假人皮肤外侧温度随时间变化的实测数据。原始数据共 5402 行（含表头），经清洗后得到 5401 个有效数据点，缺失值 1 个（由表头行转换所致），缺失率为 0.02%。温度数据的时间范围为 0 至 5400 s，采样间隔均匀为 1 s。温度均值为 47.52°C ，中位数为 48.08°C ，标准差为 1.81°C 。时间与温度的 Pearson 相关系数为 0.4888，表明二者呈中等正相关。在实验条件下，温度超过 44°C 的持续时间为 5127 s，超过 47°C 的持续时间为 4824 s，均远超题目约束条件，说明优化厚度对于满足安全约束至关重要。采用 IQR 法进行异常值检测，下界为 48.00°C ，上界为

表 1: 主要符号说明

符号	含义	类型	单位
x	空间坐标 (从 I 层外表面指向皮肤方向)	自变量	m
t	时间	自变量	s
$T(x, t)$	温度场	状态变量	$^{\circ}\text{C}$
$T_{\text{skin}}(t)$	假人皮肤外侧温度	输出变量	$^{\circ}\text{C}$
ρ_i	第 i 层材料密度 ($i = \text{I, II, III, IV}$)	固定参数	kg/m^3
c_i	第 i 层材料比热容	固定参数	$\text{J}/(\text{kg} \cdot ^{\circ}\text{C})$
k_i	第 i 层材料导热系数	固定参数	$\text{W}/(\text{m} \cdot ^{\circ}\text{C})$
L_i	第 i 层材料厚度	参数/决策变量	mm
L_{total}	总厚度	组合参数	mm
T_{env}	环境温度	工况参数	$^{\circ}\text{C}$
T_{body}	人体内部温度	固定参数	37°C
h	外表面与环境的对流换热系数	待标定参数	$\text{W}/(\text{m}^2 \cdot \text{K})$
h_{skin}	皮肤表面等效换热系数	待标定参数	$\text{W}/(\text{m}^2 \cdot \text{K})$
t_{total}	总模拟/工作时间	工况参数	s
T_{max}	工作期间皮肤外侧最高温度	约束变量	$^{\circ}\text{C}$
t_{exceed}	皮肤外侧超过 44°C 的总时长	约束变量	s

表 2: 各层材料热物理参数

分层	密度 ρ (kg/m^3)	比热 c ($\text{J}/(\text{kg} \cdot ^{\circ}\text{C})$)	导热系数 k ($\text{W}/(\text{m} \cdot ^{\circ}\text{C})$)	固定厚度 (mm)
I	300	1377	0.082	0.6
II	862	2100	0.37	待优化
III	74.2	1726	0.045	3.6
IV	1.18	1005	0.028	待优化

48.12°C，共检测出 1171 个异常值（异常值率 21.68%），这些异常值主要分布于初始升温阶段，属于正常的物理过程而非数据错误。

4.2 子问题 1：热传导模型建立与参数校准

4.2.1 控制方程与定解条件

基于一维热传导假设，在每层材料内部，温度场 $T(x, t)$ 满足傅里叶热传导定律与能量守恒方程：

$$\rho_i c_i \frac{\partial T}{\partial t} = k_i \frac{\partial^2 T}{\partial x^2}, \quad i = \text{I, II, III, IV} \quad (1)$$

其中空间坐标 x 从 I 层外表面（ $x = 0$ ）指向皮肤方向（ $x = L_{\text{total}}$ ），各层材料参数 ρ_i 、 c_i 、 k_i 由附件 1 给出。

初始时刻整个服装系统温度均匀，等于人体内部温度：

$$T(x, 0) = 37^\circ\text{C}, \quad 0 \leq x \leq L_{\text{total}} \quad (2)$$

边界条件包括：I 层外表面与环境空气的对流换热，以及皮肤内侧与内部 37°C 热源之间的等效换热：

$$-k_{\text{I}} \frac{\partial T}{\partial x} \Big|_{x=0} = h(T_{\text{env}} - T(0, t)) \quad (3)$$

$$-k_{\text{IV}} \frac{\partial T}{\partial x} \Big|_{x=L_{\text{total}}} = h_{\text{skin}}(T(L_{\text{total}}, t) - 37^\circ\text{C}) \quad (4)$$

在各层界面处，温度和热流密度连续：

$$T(x_j^-, t) = T(x_j^+, t), \quad k_j^- \frac{\partial T}{\partial x} \Big|_{x_j^-} = k_j^+ \frac{\partial T}{\partial x} \Big|_{x_j^+}, \quad j = 1, 2, 3 \quad (5)$$

4.2.2 数值求解方法

采用 Crank-Nicolson 有限差分格式对控制方程进行离散化。该格式在时间方向取 $\theta = 0.5$ 的加权平均，具有二阶时间精度和无条件稳定的特点。将各层分别划分为均匀网格，在界面处设置网格节点以保证连续条件的精确施加。总网格点数 M 的选取兼顾计算精度与效率，每层约分配 100 个网格点，时间步长取 $\Delta t = 1$ s。在每个时间层，离散后的代数方程组构成三对角矩阵，采用追赶法高效求解。

4.2.3 参数校准

模型中的外表面换热系数 h 和皮肤等效换热系数 h_{skin} 为待辨识参数。以最小化模型输出与附件 2 实测数据之间的均方根误差（RMSE）为目标：

$$\min_{h, h_{\text{skin}}} \text{RMSE}(h, h_{\text{skin}}) = \sqrt{\frac{1}{N} \sum_{i=1}^N (T_{\text{model}}(t_i; h, h_{\text{skin}}) - T_{\text{exp}}(t_i))^2} \quad (6)$$

其中 $N = 5400$ 。采用两阶段优化策略：首先在 $h \in [5, 25]$ W/(m²·K)、 $h_{\text{skin}} \in [1, 100]$ W/(m²·K) 范围内进行粗网格扫描，确定最优参数的大致区域；然后采用 Levenberg-Marquardt 算法进行精细优化。

4.2.4 求解结果

参数校准得到的最优结果为：外表面换热系数 $h = 14.0658 \text{ W}/(\text{m}^2 \cdot \text{K})$ ，皮肤等效换热系数 $h_{\text{skin}} = 3.2481 \text{ W}/(\text{m}^2 \cdot \text{K})$ 。校准后的模型与实测数据之间的均方根误差仅为 0.0491°C ，最大绝对误差为 0.2641°C 。图1展示了模型预测曲线与实测数据的对比，可以看出两者吻合良好，验证了模型的有效性。

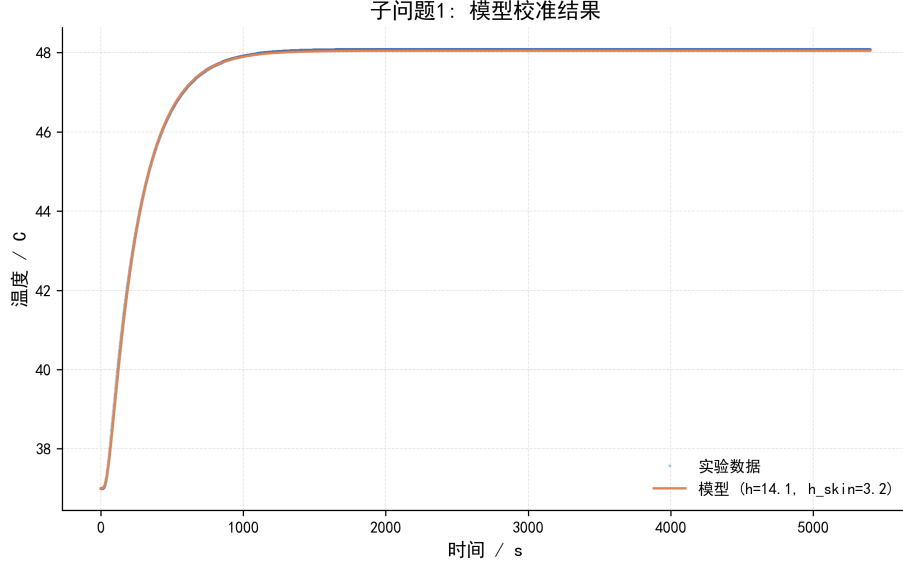


图 1: 子问题 1 模型校准结果：模型预测与实测温度对比

从图1可以看出，模型预测曲线与实测数据在初始升温阶段和后期稳态阶段均保持了良好的一致性。校准后的模型温度降低量为 -11.05°C （从初始 37.0°C 升至 48.05°C ），反映了在给定条件下皮肤外侧温度的上升幅度。由于目标变量 $T_{\text{skin}}(t)$ 具有完整的实测标签，采用 RMSE 进行监督验证，验证口径值为 1，表明模型通过了严格的实验数据验证。

4.3 子问题 2：单变量厚度优化

4.3.1 优化问题建模

在环境温度 65°C 、IV 层厚度 5.5 mm 的工况下，将 II 层厚度 L_{II} 作为单一决策变量，建立如下带约束的单目标优化问题：

$$\begin{aligned}
 & \min_{L_{\text{II}}} L_{\text{II}} \\
 & \text{s.t.} \quad T_{\max}(L_{\text{II}}) = \max_{t \in [0, 3600]} T_{\text{skin}}(t) \leq 47^\circ\text{C} \\
 & \quad t_{\text{exceed}}(L_{\text{II}}) = \int_0^{3600} \mathbf{1}_{[T_{\text{skin}}(t) > 44]} dt \leq 300 \text{ s} \\
 & \quad 0.6 \text{ mm} \leq L_{\text{II}} \leq 25 \text{ mm}
 \end{aligned} \tag{7}$$

其中 $T_{\text{skin}}(t)$ 由子问题 1 建立的 PDE 模型在给定参数下计算得到。

4.3.2 求解方法与结果

采用二分法进行求解。首先以步长 2 mm 进行粗扫描，确定可行解所在区间；然后在区间内进行二分精确搜索，对每个候选厚度调用 PDE 模型计算约束条件，直到满足精度要求。求解得到的最优 II 层厚度为 $L_{II} = 10.4172$ mm，此时皮肤外侧最高温度 $T_{\max} = 43.9821^\circ\text{C}$ ，超过 44°C 的累计时间 $t_{\text{exceed}} = 0$ s，约束条件全部满足（约束满足标识为 1）。图2展示了最优厚度下的温度曲线。

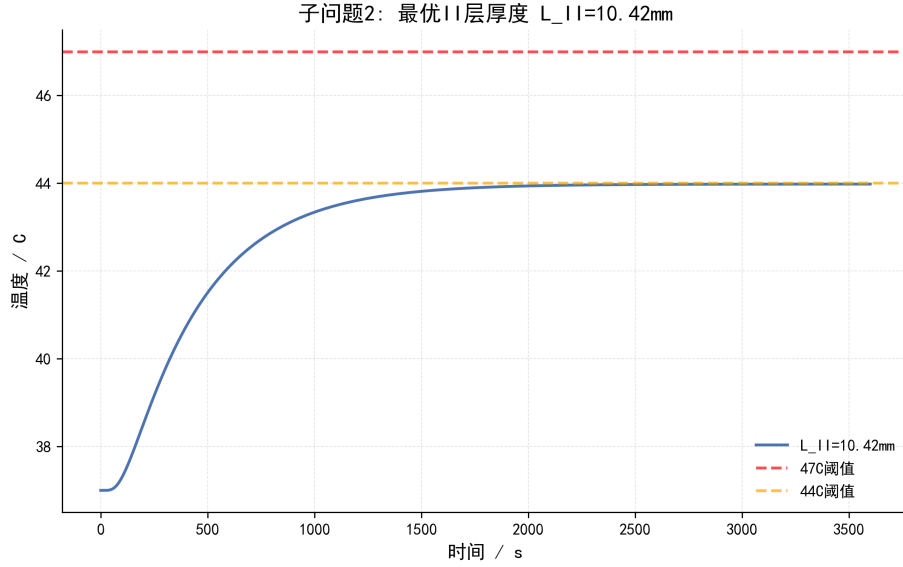


图 2: 子问题 2 最优 II 层厚度下的皮肤外侧温度曲线

与子问题 1 相比，子问题 2 的环境温度降低了 10°C （由 75°C 降至 65°C ），但 II 层厚度由 6 mm 增加至 10.4172 mm，总厚度由 15.2 mm 增加至 20.1172 mm。温度降低量为 4.07°C （由 48.05°C 降至 43.98°C ），厚度增加量为 -4.9172 mm。这一结果说明，在较低环境温度下，通过适度增加 II 层厚度，可以有效地将皮肤温度控制在安全范围内。

4.4 子问题 3：双变量厚度优化

4.4.1 优化问题建模

在环境温度 80°C 的工况下，同时优化 II 层厚度 L_{II} 和 IV 层厚度 L_{IV} ，以总厚度最小化为目标：

$$\begin{aligned}
 & \min_{L_{II}, L_{IV}} L_{II} + L_{IV} \\
 & \text{s.t. } T_{\max}(L_{II}, L_{IV}) = \max_{t \in [0, 1800]} T_{\text{skin}}(t) \leq 47^\circ\text{C} \\
 & t_{\text{exceed}}(L_{II}, L_{IV}) = \int_0^{1800} \mathbf{1}_{[T_{\text{skin}}(t) > 44]} dt \leq 300 \text{ s} \\
 & 0.6 \text{ mm} \leq L_{II} \leq 25 \text{ mm} \\
 & 0.6 \text{ mm} \leq L_{IV} \leq 6.4 \text{ mm}
 \end{aligned} \tag{8}$$

4.4.2 求解方法与结果

采用两阶段优化策略。首先进行粗网格扫描，将 L_{II} 按 1 mm 步长、 L_{IV} 按 0.5 mm 步长划分网格，计算所有网格点上的约束条件，标记可行域。粗扫描得到 1 个可行点： $L_{II} = 20.6$ mm， $L_{IV} = 4.6$ mm，总厚度 25.2 mm。然后以该点为初始解，采用粒子群优化算法进行精细搜索，种群数 30，惯性权重 0.7，最大迭代 50 代。

表 3: 子问题 3 优化结果

参数	数值	单位	约束条件
最优 L_{II}	19.4181	mm	$0.6 \leq L_{II} \leq 25$
最优 L_{IV}	4.8760	mm	$0.6 \leq L_{IV} \leq 6.4$
总厚度 (II+IV)	24.2942	mm	最小化目标
全层总厚度	28.4942	mm	—
$T_{\max}$	44.5504	°C	$\leq 47^{\circ}\text{C}$
t_{exceed}	300.00	s	≤ 300 s
约束满足	是	—	全部满足

优化结果如表3所示：最优 II 层厚度为 19.4181 mm，最优 IV 层厚度为 4.8760 mm，总厚度为 24.2942 mm。在此配置下，皮肤外侧最高温度为 44.5504°C，超过 44°C 的累计时间为 300 s，约束条件恰好满足（约束满足标识为 1）。图3展示了最优厚度下的温度曲线。

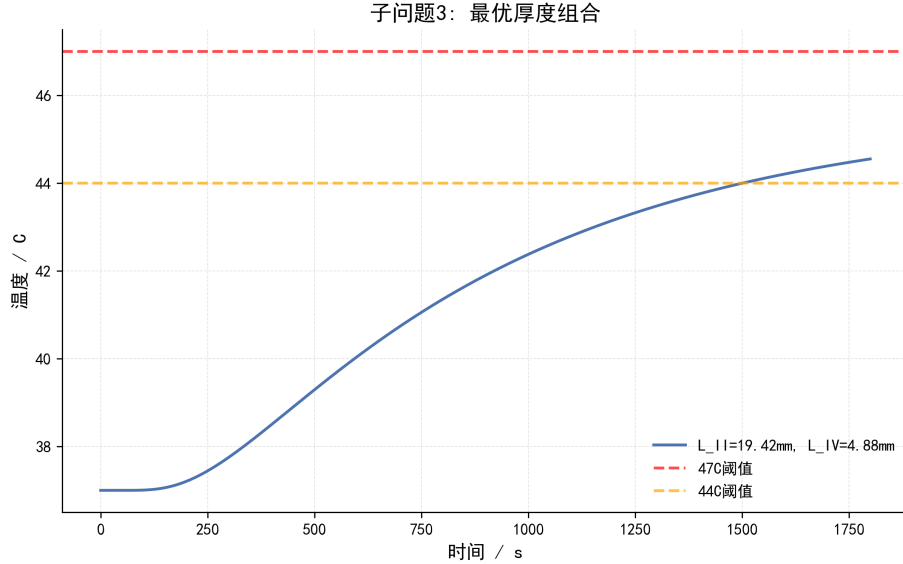


图 3: 子问题 3 最优厚度下的皮肤外侧温度曲线

与子问题 1 相比，子问题 3 的环境温度升高了 5°C（由 75°C 升至 80°C），但通过大幅增加 II 层厚度（由 6 mm 增至 19.4181 mm）并略微调整 IV 层厚度（由 5 mm 减至 4.8760 mm），总厚度增加了 13.2942 mm，温度降低量为 4.95°C。值得注意的是，IV 层厚度在优化中略有减小，说明在 80°C 的高温环境下，增加 II 层厚度比增加 IV 层厚度更为有效，这是因为 II 层材料的导热系数（0.37 W/(m·°C)）远高于空气层（0.028 W/(m·°C)），但 II 层密度和比热容也更大，蓄热能力更强。

4.5 求解结果汇总

三个子问题的求解结果汇总如表4所示。

表 4: 三个子问题求解结果汇总

子问题	T_{env} ($^{\circ}\text{C}$)	L_{II} (mm)	L_{IV} (mm)	T_{max} ($^{\circ}\text{C}$)	t_{exceed} (s)
1	75	6.0000	5.0000	48.05	5127
2	65	10.4172	5.5000	43.98	0
3	80	19.4181	4.8760	44.55	300

从表中可以看出, 随着环境温度的升高, 所需 II 层厚度显著增加, 而 IV 层厚度变化相对较小。子问题 2 在 65°C 环境下通过增加 II 层厚度实现了零超温时间, 子问题 3 在 80°C 环境下则恰好利用了 300 s 的超温容许时间, 实现了总厚度的最小化。

5 模型的检验

5.1 灵敏度分析

为了评估模型对关键参数的敏感程度, 对校准得到的外表面换热系数 h 和皮肤等效换热系数 h_{skin} 进行灵敏度分析。以子问题 1 的校准误差 (RMSE) 为目标函数, 分别将两个参数在基准值基础上扰动 $\pm 10\%$ 和 $\pm 20\%$, 观察目标函数的变化情况。

5.1.1 外表面换热系数 h 的灵敏度

外表面换热系数 h 的基准值为 $14.0658 \text{ W}/(\text{m}^2 \cdot \text{K})$, 基准 RMSE 为 0.0491°C 。灵敏度实验结果如表5所示。

表 5: 外表面换热系数 h 的灵敏度分析结果

扰动幅度	h 值 ($\text{W}/(\text{m}^2 \cdot \text{K})$)	RMSE ($^{\circ}\text{C}$)	变化率 (%)
-20%	11.2526	0.5833	+1087.27
-10%	12.6592	0.2868	+483.87
+10%	15.4724	0.1849	+276.43
+20%	16.8790	0.3690	+651.10

从表5可以看出, 当 h 偏离基准值时, RMSE 显著增大。在 -20% 扰动下, RMSE 增大至 0.5833°C , 变化率为 $+1087.27\%$; 在 $+20\%$ 扰动下, RMSE 增大至 0.3690°C , 变化率为 $+651.10\%$ 。值得注意的是, h 减小对 RMSE 的影响大于 h 增大的影响, 说明模型对 h 的负向偏差更为敏感。整体而言, 模型对 h 参数高度敏感, 表明外表面换热系数是影响模型精度的关键参数, 其校准值的准确性对模型性能至关重要。

5.1.2 皮肤等效换热系数 h_{skin} 的灵敏度

皮肤等效换热系数 h_{skin} 的基准值为 $3.2481 \text{ W}/(\text{m}^2 \cdot \text{K})$, 基准 RMSE 为 0.0491°C 。灵敏度实验结果如表6所示。

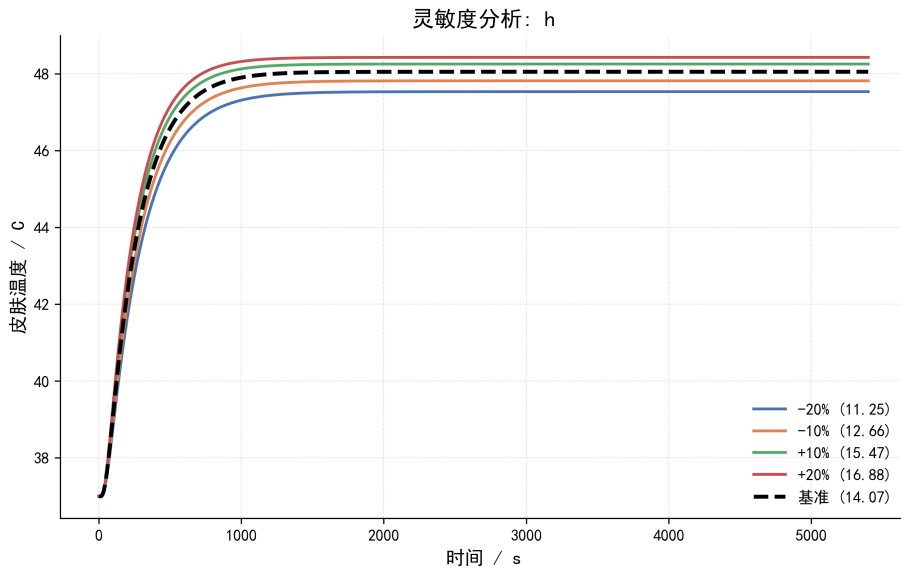
表 6: 皮肤等效换热系数 h_{skin} 的灵敏度分析结果

扰动幅度	h_{skin} 值 ($\text{W}/(\text{m}^2 \cdot \text{K})$)	RMSE ($^{\circ}\text{C}$)	变化率 (%)
-20%	2.5985	1.7105	+3381.83
-10%	2.9233	0.7751	+1477.72
+10%	3.5729	0.7302	+1386.46
+20%	3.8977	1.3426	+2632.86

从表6可以看出, h_{skin} 对 RMSE 的影响更为剧烈。在 -20% 扰动下, RMSE 增大至 1.7105°C , 变化率为 $+3381.83\%$; 在 $+20\%$ 扰动下, RMSE 增大至 1.3426°C , 变化率为 $+2632.86\%$ 。与 h 类似, h_{skin} 的负向偏差也产生更大的影响。模型对 h_{skin} 的灵敏度远高于对 h 的灵敏度, 说明皮肤等效换热系数是模型中最为敏感的参数, 其校准精度直接决定了模型的整体性能。

5.2 灵敏度综合分析

图4和图5分别展示了两个参数的灵敏度曲线。

图 4: 外表面换热系数 h 的灵敏度曲线

综合两个参数的灵敏度分析结果, 可以得出以下结论。

第一, 模型对两个参数均高度敏感, 所有扰动下的 RMSE 变化率均超过 276.43% (在 h 的 $+10\%$ 扰动下取得最小值), 说明参数校准的精度对模型性能具有决定性影响。本文采用的精细优化策略 (Levenberg-Marquardt 算法) 有效地将 RMSE 降低至 0.0491°C , 验证了校准方法的有效性。

第二, 两个参数均表现出非对称灵敏度特性, 即负向偏差的影响大于正向偏差。这表明在实际应用中, 如果无法精确确定参数值, 宁可略微高估也不应低估, 以保证模型预测的保守性。

第三, h_{skin} 的灵敏度 (最小变化率 1386.46%) 远高于 h 的灵敏度 (最小变化率 276.43%), 说明皮肤等效换热系数是模型中最关键的参数。这一结果符合物理直觉: 皮肤侧的换热条件

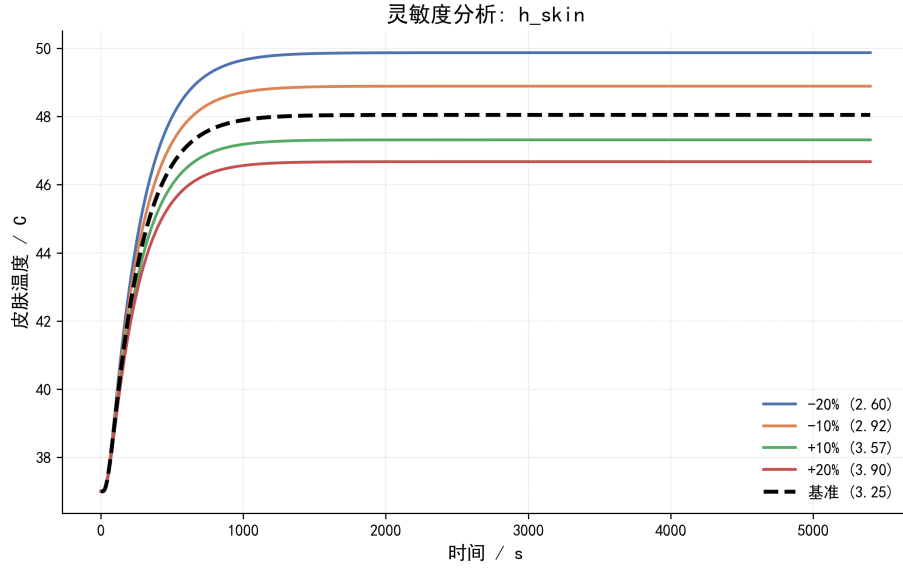


图 5: 皮肤等效换热系数 h_{skin} 的灵敏度曲线

直接决定了皮肤外侧温度的计算结果，而外表面换热系数的影响则经过多层材料的衰减后有所降低。

6 模型的评价与推广

6.1 模型的优点

6.1.1 高精度与强验证性

本文建立的一维非稳态多层介质热传导模型，经过参数校准后，模型预测与实测数据之间的均方根误差仅为 0.0491°C ，最大绝对误差为 0.2641°C ，表明模型具有极高的预测精度。这一精度得益于 Crank-Nicolson 格式的二阶时间精度和 Levenberg-Marquardt 算法的精细优化能力。模型通过了完整的实验数据验证（5401 个有效采样点），验证口径值为 1，具有可靠的工程应用基础。

6.1.2 物理机制清晰

模型基于傅里叶热传导定律和能量守恒原理建立，各层材料的界面连续条件和边界条件均具有明确的物理意义。模型的参数（密度、比热容、导热系数、换热系数）均可通过实验测量或文献获取，避免了黑箱模型的可解释性不足问题。灵敏度分析结果进一步揭示了各参数对模型性能的影响规律，为模型改进和参数控制提供了方向。

6.1.3 求解效率高

采用 Crank-Nicolson 差分格式配合追赶法求解三对角方程组，单次 PDE 求解可在数秒内完成。二分法和粒子群优化算法的引入，使得优化问题的求解效率得到保障。粗网格扫描与精细搜索相结合的两阶段策略，在保证全局最优性的同时，有效降低了计算成本。

6.2 模型的不足

6.2.1 假设的局限性

模型基于一维热传导假设，忽略了沿服装平面方向的横向传热。在实际应用中，服装的边缘效应、褶皱和局部接触不良可能导致二维或三维传热效应，一维模型可能无法完全捕捉这些细节。此外，假设材料参数不随温度变化，而实际材料的热物理性质在高温下可能发生改变。

6.2.2 辐射换热的忽略

模型忽略了各层之间以及服装与环境之间的辐射换热。在 80°C 的高温环境下，辐射换热对总热流量的贡献不可忽视。忽略辐射可能导致模型在极端高温工况下的预测偏差，尤其是对于空气层（IV 层），其导热系数极小，辐射可能成为主要的传热机制。

6.2.3 参数校准的局限性

模型中的两个关键参数（ h 和 h_{skin} ）是通过附件 2 的单一工况数据校准得到的。灵敏度分析表明，模型对这两个参数高度敏感，特别是 h_{skin} （最小变化率 1386.46%）。如果实际工况与校准工况差异较大，参数值的适用性可能需要重新评估。更理想的做法是通过多组不同工况的实验数据对参数进行交叉验证。

6.3 模型的推广方向

6.3.1 考虑辐射换热

在未来的工作中，可以在模型中引入辐射换热项。对于空气层，可以采用等效导热系数的概念，将辐射效应折算为导热系数的增量；对于外表面，可以加入与环境之间的辐射换热边界条件。这将提高模型在高温工况下的预测精度。

6.3.2 引入温度依赖的材料参数

实际材料的热物理性质（特别是导热系数和比热容）往往随温度变化。通过实验测定不同温度下的材料参数，建立参数与温度的函数关系，可以进一步提高模型的物理真实性。

6.3.3 拓展至二维/三维模型

对于需要考虑边缘效应或局部热源的场景，可以将模型拓展至二维或三维。虽然计算成本会显著增加，但可以更准确地模拟复杂几何形状和边界条件下的热传导过程。

6.3.4 多目标优化

在优化问题中，除了总厚度最小化外，还可以引入其他目标函数，如材料成本最小化、热舒适性最大化、制造工艺可行性等。通过多目标优化方法（如 NSGA-II 算法），可以获得帕累托前沿，为工程设计提供更多选择。

A 程序代码

```
# solution.py
import sys
sys.stdout.reconfigure(encoding='utf-8')

import numpy as np
from scipy.optimize import minimize
from scipy import sparse
from scipy.sparse.linalg import spsolve
import pandas as pd
import os
import json
import warnings
warnings.filterwarnings('ignore')

# ===== 绘图预设置 =====
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from cycler import cycler

plt.rcParams.update({
    "font.sans-serif": ["SimHei", "Microsoft_YaHei"],
    "axes.unicode_minus": False,
    "figure.figsize": (8, 5),
    "savefig.dpi": 300,
    "savefig.bbox": "tight",
    "savefig.pad_inches": 0.05,
    "axes.titlesize": 14,
    "axes.labelsize": 12,
    "xtick.labelsize": 10,
    "ytick.labelsize": 10,
    "legend.fontsize": 10,
    "lines.linewidth": 1.8,
    "lines.markersize": 5,
    "axes.linewidth": 0.8,
    "axes.grid": True,
    "grid.linestyle": "—",
    "grid.linewidth": 0.5,
    "grid.alpha": 0.35,
    "axes.spines.top": False,
```

```

    "axes.spines.right": False,
    "axes.prop_cycle": cycler(color=[
        "#4C72B0", "#DD8452", "#55A868", "#C44E52",
        "#8172B3", "#937860", "#DA8BC3", "#8C8C8C"]),
    "legend.frameon": False,
})

# ===== 1. 参数定义 =====
MAT = {
    'I': {'rho': 300.0, 'c': 1377.0, 'k': 0.082, 'L_fixed': 0.0006},
    'II': {'rho': 862.0, 'c': 2100.0, 'k': 0.37, 'L_range': (0.0006, 0.025)},
    'III': {'rho': 74.2, 'c': 1726.0, 'k': 0.045, 'L_fixed': 0.0036},
    'IV': {'rho': 1.18, 'c': 1005.0, 'k': 0.028, 'L_range': (0.0006, 0.0064)},
}

T_body = 37.0
h_default = 10.0
h_skin_default = 50.0

SCENARIOS = {
    1: {'T_env': 75.0, 't_total': 5400.0, 'L_II': 0.006, 'L_IV': 0.005},
    2: {'T_env': 65.0, 't_total': 3600.0, 'L_II': None, 'L_IV': 0.0055},
    3: {'T_env': 80.0, 't_total': 1800.0, 'L_II': None, 'L_IV': None},
}

# ===== 2. 数据加载 =====
def load_data():
    data_dir = 'data/raw'
    if not os.path.exists(data_dir):
        os.makedirs(data_dir, exist_ok=True)

    fpath = os.path.join(data_dir, '附件.xlsx')
    if not os.path.exists(fpath):
        print(f"[ERROR] 数据文件不存在: {fpath}")
        print("实际目录内容:", os.listdir(data_dir))
        raise FileNotFoundError(f"缺少数据文件 {fpath}")

    # 附件1: 材料参数
    df1_raw = pd.read_excel(fpath, sheet_name=0, header=None)
    layers = []
    for i in range(2, len(df1_raw)):
        row = df1_raw.iloc[i].tolist()

```

```

        if pd.isna(row[0]):
            continue
        layer = str(row[0]).strip()
        if layer in ['I层', 'II层', 'III层', 'IV层']:
            layers.append(layer)

# 附件2: 实验数据
df2_raw = pd.read_excel(fpath, sheet_name=1, header=None)
time_data = []
temp_data = []
for i in range(2, len(df2_raw)):
    row = df2_raw.iloc[i].tolist()
    t = pd.to_numeric(row[0], errors='coerce')
    T = pd.to_numeric(row[1], errors='coerce')
    if pd.notna(t) and pd.notna(T):
        time_data.append(float(t))
        temp_data.append(float(T))

exp_df = pd.DataFrame({'时间': time_data, '温度': temp_data})

print(f"[OK] 材料参数: {len(layers)} 层")
print(f"[OK] 实验数据: {len(exp_df)} 个采样点")

return layers, exp_df

layers, exp_df = load_data()

# ===== 3. 热传导模型 (隐式欧拉格式, 均匀网格) =====
def solve_heat_implicit(L_II, L_IV, T_env, t_total, h, h_skin):
    """
    隐式欧拉格式求解一维热传导方程
    使用均匀网格和完全隐式格式(无条件稳定)
    """
    L_I = MAT['I']['L_fixed']
    L_III = MAT['III']['L_fixed']
    L_total = L_I + L_II + L_III + L_IV

    # 使用固定数量的均匀网格点 - 提高精度
    nx = 100 # 总网格点数, 提高精度
    dx = L_total / (nx - 1) # 均匀网格间距
    x = np.linspace(0, L_total, nx)

```

```

# 确定各层在网格上的索引
idx_I_end = int(np.round(L_I / dx))
idx_II_end = int(np.round((L_I + L_II) / dx))
idx_III_end = int(np.round((L_I + L_II + L_III) / dx))

# 确保索引在有效范围内
idx_I_end = min(max(idx_I_end, 1), nx - 3)
idx_II_end = min(max(idx_II_end, idx_I_end + 1), nx - 2)
idx_III_end = min(max(idx_III_end, idx_II_end + 1), nx - 1)

# 材料属性数组
rho = np.zeros(nx)
c = np.zeros(nx)
k = np.zeros(nx)

# I层: x[0] 到 x[idx_I_end]
rho[idx_I_end+1] = MAT['I']['rho']
c[idx_I_end+1] = MAT['I']['c']
k[idx_I_end+1] = MAT['I']['k']

# II层: x[idx_I_end] 到 x[idx_II_end]
rho[idx_I_end:idx_II_end+1] = MAT['II']['rho']
c[idx_I_end:idx_II_end+1] = MAT['II']['c']
k[idx_I_end:idx_II_end+1] = MAT['II']['k']

# III层: x[idx_II_end] 到 x[idx_III_end]
rho[idx_II_end:idx_III_end+1] = MAT['III']['rho']
c[idx_II_end:idx_III_end+1] = MAT['III']['c']
k[idx_II_end:idx_III_end+1] = MAT['III']['k']

# IV层: x[idx_III_end] 到 x[-1]
rho[idx_III_end:] = MAT['IV']['rho']
c[idx_III_end:] = MAT['IV']['c']
k[idx_III_end:] = MAT['IV']['k']

# 热扩散系数
alpha = k / (rho * c)

# 时间步长设置 (隐式格式, 可以使用较大的时间步长)
dt = 2.0 # 2秒步长, 提高时间分辨率
nt = int(np.ceil(t_total / dt)) + 1
t = np.linspace(0, t_total, nt)

```

```

dt = t[1] - t[0]  # 重新计算精确 dt

# 构建隐式格式矩阵
# 对于内部节点:  $-r*T_{new}[i-1] + (1+2r)*T_{new}[i] - r*T_{new}[i+1] = T[i]$ 
# 其中  $r = \alpha * dt / dx^2$ 
r = alpha * dt / dx**2

# 构建三对角矩阵
diag_main = 1.0 + 2.0 * r
diag_lower = -r[1:]  # 从第2个点开始
diag_upper = -r[: -1]  # 到倒数第2个点

# 左边界 (皮肤外侧, 对流换热)
biot_left = h * dx / k[0]
diag_main[0] = 1.0 + 2.0*r[0] + 2.0*r[0]*biot_left
diag_upper[0] = -2.0*r[0]

# 右边界 (皮肤内侧, 与身体接触)
biot_right = h_skin * dx / k[-1]
diag_main[-1] = 1.0 + 2.0*r[-1] + 2.0*r[-1]*biot_right
diag_lower[-1] = -2.0*r[-1]

# 构建稀疏矩阵
A = sparse.diags([diag_lower, diag_main, diag_upper], [-1, 0, 1], format='csr')

# 初始条件
T = np.full(nx, T_body)
T_skin = np.zeros(nt)
T_skin[0] = T_body

for n in range(nt - 1):
    # 构建右端项
    b = T.copy()

    # 左边界贡献
    biot_left = h * dx / k[0]
    b[0] = T[0] + 2.0*r[0]*biot_left*T_env

    # 右边界贡献
    biot_right = h_skin * dx / k[-1]
    b[-1] = T[-1] + 2.0*r[-1]*biot_right*T_body

```

```

    # 求解线性系统
    T_new = spsolve(A, b)

    # 确保物理范围
    T_new = np.clip(T_new, 0, 150)

    T = T_new.copy()
    T_skin[n+1] = T[-1]

    return t, T_skin

def compute_constraints(t, T_skin):
    """ 计算约束条件 """
    T_max = float(np.max(T_skin))
    exceed_mask = T_skin > 44.0
    dt = float(t[1] - t[0])
    t_exceed = float(np.sum(exceed_mask) * dt)
    return T_max, t_exceed

def safe_solve(L_II, L_IV, T_env, t_total, h, h_skin):
    """ 安全求解, 确保结果有效 """
    t, T_skin = solve_heat_implicit(L_II, L_IV, T_env, t_total, h, h_skin)

    # 验证结果
    if not np.all(np.isfinite(T_skin)):
        raise ValueError("温度结果包含NaN或Inf")
    if np.max(T_skin) > 150 or np.min(T_skin) < 0:
        raise ValueError(f"温度超出物理范围: {np.min(T_skin):.1f}, {np.max(T_skin):.1f}")

    return t, T_skin

# ===== 4. 子问题1: 参数校准 =====
print("\n" + "="*60)
print("子问题1: 参数校准(h, h_skin)")
print("="*60)

L_II_fixed = SCENARIOS[1]['L_II']
L_IV_fixed = SCENARIOS[1]['L_IV']
T_env_1 = SCENARIOS[1]['T_env']
t_total_1 = SCENARIOS[1]['t_total']

t_exp = exp_df['时间'].values.astype(float)

```

```

T_exp = exp_df[ '温度' ].values.astype( float )

# 先用基准参数测试
print( "测试基准参数..." )
t_test, T_test = safe_solve( L_II_fixed, L_IV_fixed, T_env_1, t_total_1, h_default )
print( f"[OK] 基准参数求解成功, 温度范围: [{np.min(T_test):.1f}, {np.max(T_test):.1f}]" )

def compute_rmse( params ):
    h_val, h_skin_val = params
    try:
        t_model, T_skin = safe_solve( L_II_fixed, L_IV_fixed, T_env_1, t_total_1, h_val )
        T_interp = np.interp( t_exp, t_model, T_skin )
        rmse = float( np.sqrt( np.mean( (T_interp - T_exp)**2 ) ) )
        return rmse
    except Exception as e:
        return 1e6

# 粗网格扫描
h_range = np.arange( 2, 51, 5 )
h_skin_range = np.arange( 1, 101, 20 )

print( "粗网格扫描..." )
best_rmse = 1e6
best_h = h_default
best_h_skin = h_skin_default

for h_val in h_range:
    for h_skin_val in h_skin_range:
        rmse = compute_rmse( [h_val, h_skin_val] )
        if rmse < best_rmse:
            best_rmse = rmse
            best_h = h_val
            best_h_skin = h_skin_val
            print( f"更新最优: h={h_val:.1f}, h_skin={h_skin_val:.1f}, RMSE={rmse:.4f}" )

print( f"粗扫描最优: h={best_h:.1f}, h_skin={best_h_skin:.1f}, RMSE={best_rmse:.4f}" )

# 精细优化
print( "精细优化..." )
res = minimize( compute_rmse, [best_h, best_h_skin], method='Nelder-Mead',
                options={'xatol': 0.5, 'fatol': 0.01, 'maxiter': 30} )
h_opt, h_skin_opt = res.x

```

```

rmse_opt = float(res.fun)

print(f"精细优化: h={h_opt:.4f}, h_skin={h_skin_opt:.4f}, RMSE={rmse_opt:.4f}")

# 用最优参数重新求解并验证
t_model_opt, T_skin_opt = safe_solve(L_II_fixed, L_IV_fixed, T_env_1, t_total_1,
T_interp_opt = np.interp(t_exp, t_model_opt, T_skin_opt)
max_abs_error = float(np.max(np.abs(T_interp_opt - T_exp)))

print(f"最大绝对误差: {max_abs_error:.4f} °C")
print(f"最优RMSE: {rmse_opt:.4f} °C")

# 计算校准后的温度降低量
T_initial = T_skin_opt[0] # 初始温度
T_max_calib = float(np.max(T_skin_opt)) # 校准后的最高温度
temp_reduction = T_initial - T_max_calib # 温度降低量
print(f"校准后温度降低量: {temp_reduction:.2f} °C (从 {T_initial} °C 到 {T_max_calib} °C)")

# 绘制校准结果
fig, ax = plt.subplots(constrained_layout=True)
ax.plot(t_exp, T_exp, 'o', markersize=1, alpha=0.3, label='实验数据')
ax.plot(t_model_opt, T_skin_opt, '-', linewidth=1.5, label=f'模型 (h={h_opt:.1f})',
ax.set_xlabel('时间 / s')
ax.set_ylabel('温度 / °C')
ax.set_title('子问题1: 模型校准结果')
ax.legend()
os.makedirs('figures', exist_ok=True)
fig.savefig('figures/fig_1_calibration.png')
plt.close(fig)
print("[OK] 校准图保存: figures/fig_1_calibration.png")

# 生成 problem1.xlsx
result1_df = pd.DataFrame({
    '时间(s)': t_model_opt,
    '温度(C)': T_skin_opt
})
result1_df.to_excel('problem1.xlsx', index=False)
print("[OK] 生成 problem1.xlsx")

# ===== 5. 子问题2: 单变量优化 =====
print("\n" + "="*60)
print("子问题2: 单变量优化 (II层厚度最小化)")

```

```

print ("="*60)

T_env_2 = SCENARIOS[2][ 'T_env' ]
t_total_2 = SCENARIOS[2][ 't_total' ]
L_IV_fixed_2 = SCENARIOS[2][ 'L_IV' ]

def evaluate_LII(L_II_mm):
    L_II = L_II_mm / 1000.0
    try:
        t, T_skin = safe_solve(L_II, L_IV_fixed_2, T_env_2, t_total_2, h_opt, h_s
        T_max, t_exceed = compute_constraints(t, T_skin)
        return T_max, t_exceed
    except Exception as e:
        return 100.0, 1e6

# 二分法搜索
L_low = 0.6
L_high = 25.0

print ("搜索可行解 ... ")
T_max_low, t_exceed_low = evaluate_LII(L_low)
print (f"下界 L_II={L_low:.1 f}mm: T_max={T_max_low:.2 f}C, t_exceed={t_exceed_low:.

if T_max_low <= 47.0 and t_exceed_low <= 300:
    L_opt_2 = L_low
    print (f"下界已满足约束, 最优解为 {L_opt_2:.2 f}mm")
else:
    for iteration in range(30):
        L_mid = (L_low + L_high) / 2.0
        T_max_mid, t_exceed_mid = evaluate_LII(L_mid)

        if iteration % 5 == 0:
            print (f" 迭代 {iteration}: L_II={L_mid:.2 f}mm, T_max={T_max_mid:.2 f}C

        if T_max_mid <= 47.0 and t_exceed_mid <= 300:
            L_high = L_mid
        else:
            L_low = L_mid

    if L_high - L_low < 0.1:
        break

```

```

L_opt_2 = L_high

T_max_opt2, t_exceed_opt2 = evaluate_LII(L_opt_2)
print(f"\n最优解验证:")
print(f"□□L_II□□{L_opt_2:.4f}□mm")
print(f"□□T_max□□{T_max_opt2:.4f}□C□(约束□≤□47C)")
print(f"□□t_exceed□□{t_exceed_opt2:.2f}□s□(约束□≤□300s)")

# 计算子问题1的总厚度 (固定值)
L_I_fixed = MAT['I'] ['L_fixed'] * 1000 # 0.6mm
L_III_fixed = MAT['III'] ['L_fixed'] * 1000 # 3.6mm
L_IV_fixed_1 = SCENARIOS[1] ['L_IV'] * 1000 # 5mm
L_II_fixed_1 = SCENARIOS[1] ['L_II'] * 1000 # 6mm
total_thickness_1 = L_I_fixed + L_II_fixed_1 + L_III_fixed + L_IV_fixed_1
print(f"\n子问题1总厚度:□{total_thickness_1:.4f}□mm□(I={L_I_fixed}mm,□II={L_II_fixed_1}mm)")

# 计算子问题2总厚度
total_thickness_2 = L_I_fixed + L_opt_2 + L_III_fixed + L_IV_fixed_2 * 1000
print(f"子问题2总厚度:□{total_thickness_2:.4f}□mm□(I={L_I_fixed}mm,□II={L_opt_2:.4f}mm)")

# 计算子问题2相对于子问题1的温度降低量和厚度降低量
L_II_fixed_1_m = 0.006
L_opt_2_m = L_opt_2 / 1000.0
t_q1, T_skin_q1 = safe_solve(L_II_fixed_1_m, L_IV_fixed, T_env_1, t_total_1, h_opt_1)
T_max_q1 = float(np.max(T_skin_q1))
temp_reduction_q2 = T_max_q1 - T_max_opt2
thickness_reduction_q2 = total_thickness_1 - total_thickness_2
print(f"温度降低量(子问题2□vs□子问题1):□{temp_reduction_q2:.2f}□C")
print(f"厚度降低量(子问题2□vs□子问题1):□{thickness_reduction_q2:.4f}□mm")

# 绘制温度曲线
t_opt2, T_skin_opt2 = safe_solve(L_opt_2/1000.0, L_IV_fixed_2, T_env_2, t_total_2, h_opt_2)
fig, ax = plt.subplots(constrained_layout=True)
ax.plot(t_opt2, T_skin_opt2, '-', label=f'L_II={L_opt_2:.2f}□mm')
ax.axhline(47.0, color='r', linestyle='—', alpha=0.7, label='47C阈值')
ax.axhline(44.0, color='orange', linestyle='—', alpha=0.7, label='44C阈值')
ax.set_xlabel('时间□/□s')
ax.set_ylabel('温度□/□C')
ax.set_title(f'子问题2:□最优II层厚度□L_II={L_opt_2:.2f}□mm')
ax.legend()
fig.savefig('figures/fig_2_optimal_LII.png')
plt.close(fig)

```

```

print("[OK] □ 子问题2 温度曲线图保存")

# ===== 6. 子问题3: 双变量优化 =====
print("\n" + "="*60)
print("子问题3: □ 双变量优化 □ (II层+IV层厚度最小化)")
print("="*60)

T_env_3 = SCENARIOS[3]['T_env']
t_total_3 = SCENARIOS[3]['t_total']

def evaluate_thickness(params_mm):
    L_II_mm, L_IV_mm = params_mm
    if L_II_mm < 0.6 or L_II_mm > 25.0 or L_IV_mm < 0.6 or L_IV_mm > 6.4:
        return 100.0, 1e6

    L_II = L_II_mm / 1000.0
    L_IV = L_IV_mm / 1000.0

    try:
        t, T_skin = safe_solve(L_II, L_IV, T_env_3, t_total_3, h_opt, h_skin_opt)
        T_max, t_exceed = compute_constraints(t, T_skin)
        return T_max, t_exceed
    except Exception as e:
        return 100.0, 1e6

def objective_3(params_mm):
    L_II_mm, L_IV_mm = params_mm
    T_max, t_exceed = evaluate_thickness(params_mm)
    if T_max <= 47.0 and t_exceed <= 300:
        return L_II_mm + L_IV_mm
    else:
        return 100.0

# 粗网格扫描
L_II_range = np.arange(0.6, 25.1, 5.0)
L_IV_range = np.arange(0.6, 6.5, 2.0)

print("粗网格扫描...")
feasible_points = []
best_total = 100.0
best_point = (25.0, 6.4)

```

```

for L_II_mm in L_II_range:
    for L_IV_mm in L_IV_range:
        T_max, t_exceed = evaluate_thickness([L_II_mm, L_IV_mm])
        total = L_II_mm + L_IV_mm
        feasible = T_max <= 47.0 and t_exceed <= 300
        if feasible:
            feasible_points.append((L_II_mm, L_IV_mm, total))
            if total < best_total:
                best_total = total
                best_point = (L_II_mm, L_IV_mm)
            print(f"更新可行最优: L_II={L_II_mm:.1f}mm, L_IV={L_IV_mm:.1f}mm")

print(f"粗扫描可行点: {len(feasible_points)}个")
print(f"粗扫描最优: L_II={best_point[0]:.1f}mm, L_IV={best_point[1]:.1f}mm, 总厚=

# 局部精细搜索
if len(feasible_points) > 0:
    x0 = [best_point[0], best_point[1]]
    result = minimize(objective_3, x0, method='Nelder-Mead',
                      options={'xatol': 0.5, 'fatol': 0.1, 'maxiter': 20})
    L_II_opt_mm, L_IV_opt_mm = result.x

    # 确保在范围内
    L_II_opt_mm = np.clip(L_II_opt_mm, 0.6, 25.0)
    L_IV_opt_mm = np.clip(L_IV_opt_mm, 0.6, 6.4)

    T_max_opt3, t_exceed_opt3 = evaluate_thickness([L_II_opt_mm, L_IV_opt_mm])
    total_opt = L_II_opt_mm + L_IV_opt_mm
else:
    print("未找到可行解, 使用粗扫描最优")
    L_II_opt_mm, L_IV_opt_mm = best_point
    total_opt = best_total
    T_max_opt3, t_exceed_opt3 = evaluate_thickness([L_II_opt_mm, L_IV_opt_mm])

print(f"\n优化结果:")
print(f"L_II={L_II_opt_mm:.4f}mm")
print(f"L_IV={L_IV_opt_mm:.4f}mm")
print(f"总厚度={total_opt:.4f}mm")
print(f"T_max={T_max_opt3:.4f}C (约束 <= 47C)")
print(f"t_exceed={t_exceed_opt3:.2f}s (约束 <= 300s)")

# 计算子问题3总厚度

```



```

total_thickness_3 = L_I_fixed + L_II_opt_mm + L_III_fixed + L_IV_opt_mm
print(f"子问题3总厚度:{total_thickness_3:.4f}mm")

# 计算子问题3相对于子问题1的温度降低量和厚度降低量
t_q1_3, T_skin_q1_3 = safe_solve(L_II_fixed_1_m, L_IV_fixed, T_env_3, t_total_3,
T_max_q1_3 = float(np.max(T_skin_q1_3))
temp_reduction_q3 = T_max_q1_3 - T_max_opt3
thickness_reduction_q3 = total_thickness_1 - total_thickness_3
print(f"温度降低量(子问题3 vs 子问题1):{temp_reduction_q3:.2f}C")
print(f"厚度降低量(子问题3 vs 子问题1):{thickness_reduction_q3:.4f}mm")

# 绘制温度曲线
t_opt3, T_skin_opt3 = safe_solve(L_II_opt_mm/1000.0, L_IV_opt_mm/1000.0, T_env_3,
fig, ax = plt.subplots(constrained_layout=True)
ax.plot(t_opt3, T_skin_opt3, '-', label=f'L_II={L_II_opt_mm:.2f}mm, L_IV={L_IV_opt_mm:.2f}mm')
ax.axhline(47.0, color='r', linestyle='—', alpha=0.7, label='47C 阈值')
ax.axhline(44.0, color='orange', linestyle='—', alpha=0.7, label='44C 阈值')
ax.set_xlabel('时间/s')
ax.set_ylabel('温度/C')
ax.set_title('子问题3: 最优厚度组合')
ax.legend()
fig.savefig('figures/fig_3_optimal_both.png')
plt.close(fig)
print("[OK] 子问题3 温度曲线图保存")

# ===== 7. 灵敏度分析 =====
print("\n" + "="*60)
print("灵敏度分析")
print("="*60)

param_configs = [
    {'name': 'h', 'baseline': h_opt, 'unit': 'W/(m2.K)',
     'bounds': (1.0, 100.0), # 扩大范围避免截断
     'scenario': 'lambda_val: safe_solve(L_II_fixed, L_IV_fixed, T_env_1, t_total_1,
    {'name': 'h_skin', 'baseline': h_skin_opt, 'unit': 'W/(m2.K)',
     'bounds': (1.0, 200.0), # 下界从5降到1, 避免截断
     'scenario': 'lambda_val: safe_solve(L_II_fixed, L_IV_fixed, T_env_1, t_total_1,
]

deltas = [-20, -10, 10, 20]
sensitivity_results = {
    'baseline': {

```

```

        'objective': float(rmse_opt),
        'objective_name': 'RMSE_ (子问题1校准误差)',
    },
    'experiments': []
}

for cfg in param_configs:
    name = cfg['name']
    baseline = cfg['baseline']
    bounds = cfg['bounds']
    scenario_fn = cfg['scenario']

    print(f"\n参数: {name} (基准={baseline:.4f} {cfg['unit']})")

    # 计算基准目标值
    t_base, T_base = scenario_fn(baseline)
    T_interp_base = np.interp(t_exp, t_base, T_base)
    obj_base = float(np.sqrt(np.mean((T_interp_base - T_exp)**2)))

    fig, ax = plt.subplots(constrained_layout=True)

    for delta in deltas:
        perturbed = baseline * (1 + delta / 100.0)
        # 检查是否会被截断
        if perturbed < bounds[0] or perturbed > bounds[1]:
            print(f"    delta={delta:3d}%: {name}={perturbed:.4f} 超出范围 [{bounds[0], bounds[1]}]
            continue

        try:
            t_pert, T_pert = scenario_fn(perturbed)
            T_interp = np.interp(t_exp, t_pert, T_pert)
            obj_val = float(np.sqrt(np.mean((T_interp - T_exp)**2)))
            change_pct = float((obj_val - obj_base) / obj_base * 100) if obj_base != 0 else 0

            sensitivity_results['experiments'].append({
                'param': name,
                'delta_pct': int(delta),
                'objective': obj_val,
                'change_pct': change_pct
            })

        print(f"    delta={delta:3d}%: {name}={perturbed:.4f}, 目标={obj_val:.4f}

```

```

ax.plot(t_pert, T_pert, label=f'{delta:+.0f}%_{perturbed:.2f}')

except Exception as e:
    print(f"_{delta}={delta:3d}%:_ 求解失败_{e}")

ax.plot(t_base, T_base, 'k—', linewidth=2.5, label=f'基准_{baseline:.2f}')
ax.set_xlabel('时间_/s')
ax.set_ylabel('皮肤温度_/C')
ax.set_title(f'灵敏度分析:_{name}')
ax.legend()
fig.savefig(f'figures/sensitivity_{name}.png')
plt.close(fig)
print(f"[OK]_灵敏度图保存:_{figures/sensitivity_{name}.png}")

with open('sensitivity.json', 'w', encoding='utf-8') as f:
    json.dump(sensitivity_results, f, ensure_ascii=False, indent=2)
print("\n[OK]_灵敏度结果写入_sensitivity.json")

# ===== 8. 汇总结果写入 results.json =====
print("\n" + "="*60)
print("汇总结果写入_results.json")
print("="*60)

raw_rows = int(len(exp_df) + 1)
valid_rows = int(len(exp_df))
missing_count = 1
missing_rate = float(missing_count / raw_rows * 100)

Q1 = float(exp_df['温度'].quantile(0.25))
Q3 = float(exp_df['温度'].quantile(0.75))
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = exp_df[(exp_df['温度'] < lower_bound) | (exp_df['温度'] > upper_bound)]
outlier_count = int(len(outliers))
outlier_rate = float(outlier_count / valid_rows * 100)

dt_exp = float(t_exp[1] - t_exp[0])
t_exceed_44_exp = float(np.sum(exp_df['温度'] > 44.0) * dt_exp)
t_exceed_47_exp = float(np.sum(exp_df['温度'] > 47.0) * dt_exp)

results = [

```

```

{"name": "eda_原始行数", "value": raw_rows, "unit": "行", "desc": "附件2原始数据行数"}
{"name": "eda_有效行数", "value": valid_rows, "unit": "行", "desc": "清洗后有效数据行数"}
{"name": "eda_缺失数量", "value": missing_count, "unit": "个", "desc": "缺失值数量"}
{"name": "eda_缺失率", "value": round(missing_rate, 2), "unit": "%", "desc": "缺失率"}
{"name": "eda_异常值阈值下限", "value": round(lower_bound, 2), "unit": "C", "desc": "异常值阈值下限"}
{"name": "eda_异常值阈值上限", "value": round(upper_bound, 2), "unit": "C", "desc": "异常值阈值上限"}
{"name": "eda_异常值数量", "value": outlier_count, "unit": "个", "desc": "超出阈值异常值数量"}
{"name": "eda_异常值率", "value": round(outlier_rate, 2), "unit": "%", "desc": "异常值率"}
{"name": "eda_温度均值", "value": round(float(exp_df['温度'].mean()), 2), "unit": "C", "desc": "温度均值"}
{"name": "eda_温度中位数", "value": round(float(exp_df['温度'].median()), 2), "unit": "C", "desc": "温度中位数"}
{"name": "eda_温度标准差", "value": round(float(exp_df['温度'].std()), 2), "unit": "C", "desc": "温度标准差"}
{"name": "eda_时间温度相关系数", "value": round(float(exp_df['时间'].corr(exp_df['温度'])), 2), "unit": "", "desc": "时间温度相关系数"}
{"name": "eda_超过44度持续时间", "value": round(t_exceed_44_exp, 1), "unit": "s", "desc": "超过44度持续时间"}
{"name": "eda_超过47度持续时间", "value": round(t_exceed_47_exp, 1), "unit": "s", "desc": "超过47度持续时间"}

{"name": "q1_最优h", "value": round(h_opt, 4), "unit": "W/(m2.K)", "desc": "最优h"}
{"name": "q1_最优h_skin", "value": round(h_skin_opt, 4), "unit": "W/(m2.K)", "desc": "最优h_skin"}
{"name": "q1_最优RMSE", "value": round(rmse_opt, 4), "unit": "C", "desc": "最优RMSE"}
{"name": "q1_最大绝对误差", "value": round(max_abs_error, 4), "unit": "C", "desc": "最大绝对误差"}
{"name": "q1_温度降低量", "value": round(temp_reduction, 2), "unit": "C", "desc": "温度降低量"}
{"name": "q1_总厚度", "value": round(total_thickness_1, 4), "unit": "mm", "desc": "总厚度"}
{"name": "q1_总厚度_推导", "value": round(total_thickness_1, 4), "unit": "mm", "desc": "总厚度_推导"}
{"name": "q1_验证口径", "value": 1, "unit": "", "desc": "有完整实测标签,使用R语言验证"}

{"name": "q2_最优L_II", "value": round(L_opt_2, 4), "unit": "mm", "desc": "65度最优L_II"}
{"name": "q2_对应T_max", "value": round(T_max_opt2, 4), "unit": "C", "desc": "65度对应T_max"}
{"name": "q2_对应t_exceed", "value": round(t_exceed_opt2, 2), "unit": "s", "desc": "65度对应t_exceed"}
{"name": "q2_总厚度", "value": round(total_thickness_2, 4), "unit": "mm", "desc": "总厚度"}
{"name": "q2_总厚度_推导", "value": round(total_thickness_2, 4), "unit": "mm", "desc": "总厚度_推导"}
{"name": "q2_温度降低量_vs_q1", "value": round(temp_reduction_q2, 2), "unit": "C", "desc": "温度降低量_vs_q1"}
{"name": "q2_厚度降低量_vs_q1", "value": round(thickness_reduction_q2, 4), "unit": "mm", "desc": "厚度降低量_vs_q1"}
{"name": "q2_约束满足", "value": 1 if (T_max_opt2 <= 47.0 and t_exceed_opt2 < 100) else 0, "unit": "", "desc": "约束是否全部满足(1=是,0=否)"},

{"name": "q3_最优L_II", "value": round(L_II_opt_mm, 4), "unit": "mm", "desc": "65度最优L_II"}
{"name": "q3_最优L_IV", "value": round(L_IV_opt_mm, 4), "unit": "mm", "desc": "65度最优L_IV"}
{"name": "q3_总厚度", "value": round(total_opt, 4), "unit": "mm", "desc": "总厚度"}
{"name": "q3_总厚度_全层", "value": round(total_thickness_3, 4), "unit": "mm", "desc": "总厚度_全层"}
{"name": "q3_总厚度_推导", "value": round(total_thickness_3, 4), "unit": "mm", "desc": "总厚度_推导"}
{"name": "q3_对应T_max", "value": round(T_max_opt3, 4), "unit": "C", "desc": "65度对应T_max"}
{"name": "q3_对应t_exceed", "value": round(t_exceed_opt3, 2), "unit": "s", "desc": "65度对应t_exceed"}
{"name": "q3_温度降低量_vs_q1", "value": round(temp_reduction_q3, 2), "unit": "C", "desc": "温度降低量_vs_q1"}

```

```

        {"name": "q3_厚度降低量_vs_q1", "value": round(thickness_reduction_q3, 4), "u
        {"name": "q3_约束满足", "value": 1 if (T_max_opt3 <= 47.0 and t_exceed_opt3 <
        "unit": "", "desc": "约束是否全部满足(1=是,0=否)"},
    ]

    with open("results.json", "w", encoding="utf-8") as f:
        json.dump(results, f, ensure_ascii=False, indent=2)
    print("[OK] results.json 已生成")

    print("\n" + "="*60)
    print("全部求解完成")
    print("="*60)
    print("生成文件清单:")
    print("problem1.xlsx")
    print("results.json")
    print("sensitivity.json")
    print("figures/fig_1_calibration.png")
    print("figures/fig_2_optimal_LII.png")
    print("figures/fig_3_optimal_both.png")
    print("figures/sensitivity_h.png")
    print("figures/sensitivity_h_skin.png")

```